

Normalization 02

Task 1: First Normal Form (1NF) Analysis

Given the following un-normalized flat table that was used before the EMS database was designed

EMP_RAW (emp_id, full_name, phone_numbers, dept_name, dept_location, job_titles, salary)

a) Identify every violation of 1NF in this table and explain each one clearly)

phone_nubers.1

.is multi-valued. stores two phone numbers in one field

job_title.2

.is multi-valued. An employee can hold several job titles stored as a comma-separated list

dept_name + dept_location.3

redundant data. Every employee in the IT department stores the same values "IT" and "Muscat" over and over. This is a repeating group — data about a department being embedded and duplicated inside every employee row rather than stored once in its own entity

full_name.4

is a composite, Combining first name and last name into one column means neither part can be used independently

b) Rewrite the table in 1NF. Show the resulting table(s) with at least 3 rows of sample data)

EMPLOYEE.1

(emp_id, fname, lname, salary, dept_id)

dept_id	salary	lname	fname	emp_id
10	1200	Hassan	Ali	1
20	1500	Noor	Sara	2
10	1300	Said	Omar	3

DEPARTMENT.2

(dept_id, dept_name, dept_location)

dept_location	dept_name	dept_id
Muscat	HR	10
Sohar	Finance	20

EMP_PHONE.3
(emp_id, phone_number)

phone_number	emp_id
99887766	1
99775544	1
91234567	2

EMP_JOB.4
(emp_id, job_title)

job_title	emp_id
Recruiter	1
Accountant	2
Analyst	3
Auditor	3

c) Check your own EMS schema (from Section 01) — confirm that all 6 tables satisfy 1NF) and explain why

JOB_DEPARTMENT---(job_dept , name, description , salary_range)holds one single text.1
.value , and job_ID is unique PK

SALARY_BOUNDS---(amount , annul , bonus)No multi-valued, salary_ID is PK and job_ID.2
.is FK

EMPLOYEE---(fname , lname, gender, age, contact_add, emp_email , emp_pass).3
.full_name was split into fname + lname, emp_ID is PK and job_ID , salary_id are FK

(QUALIFICATION---(position , date_in , requirements.4
PK is qual_ID and emp_ID is FK.treated as one atomic descriptive text value (VARCHAR2),
.not a list

LEAVE---(reason , date)date is one DATE value, reason is one VARCHAR2 string ,No.5
.multi-valued: one leave record per row, not a list of dates. emp_ID FK and PK is leave_ID

PAYROL-----(date , report , total_amount) every column holds exactly one value — no lists.6
.anywhere ,No multi-valued: each FK references one row in its parent table ,PK: payroll_ID

SQL DML — INSERT / UPDATE / DELETE / SELECT 03

Task 1: Populate All Tables with Seed Data Easy 1 pts

:Write INSERT statements to populate the database with realistic seed data

At least 5 departments, 10 employees, 5 salary/bonus records, 8 payroll records, 5 leave • records, 5

.qualification records

Use Oracle sequences (CREATE SEQUENCE ... START WITH 1 INCREMENT BY 1) to • generate PKs for all

.tables

.Ensure referential integrity — do not insert child records before parent records •

TABLE 1: JOB_DEPARTMENT -----

```
) CREATE TABLE JOB_DEPARTMENT
,job_ID    NUMBER(10)  CONSTRAINT pk_job_dept  PRIMARY KEY
,job_dept  VARCHAR2(100) NOT NULL
,name      VARCHAR2(150) NOT NULL
,(description VARCHAR2(500
(salary_range VARCHAR2(50
;{(
```

TABLE 2: SALARY_BONUS -----

```
) CREATE TABLE SALARY_BONUS
,salary_ID NUMBER(10)  CONSTRAINT pk_salary_bonus PRIMARY KEY
amount     NUMBER(12,2) NOT NULL
,(CONSTRAINT chk_salary_amount CHECK (amount > 0
,annual    NUMBER(12,2) NOT NULL
,bonus     NUMBER(12,2) NOT NULL
,job_ID    NUMBER(10)  NOT NULL
CONSTRAINT fk_salary_job
(FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT (job_ID
;{(
```

TABLE 3: EMPLOYEE -----

```
) CREATE TABLE EMPLOYEE
,emp_ID    NUMBER(10)  CONSTRAINT pk_employee  PRIMARY KEY
,fname     VARCHAR2(100) NOT NULL
,lname     VARCHAR2(100) NOT NULL
gender     CHAR(1)     NOT NULL
,((CONSTRAINT chk_emp_gender CHECK (gender IN ('M','F
,age       NUMBER(3)   NOT NULL
,contact_add VARCHAR2(300) NOT NULL
,emp_email VARCHAR2(200) NOT NULL
```

```
,emp_pass VARCHAR2(255) NOT NULL
,job_ID NUMBER(10) NOT NULL
,salary_ID NUMBER(10) NOT NULL
CONSTRAINT fk_emp_job
,(FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT (job_ID
CONSTRAINT fk_emp_salary
(FOREIGN KEY (salary_ID) REFERENCES SALARY_BONUS (salary_ID
;{
```

TABLE 4: QUALIFICATION -----

```
) CREATE TABLE QUALIFICATION
,qual_ID NUMBER(10) CONSTRAINT pk_qualification PRIMARY KEY
,position VARCHAR2(150) NOT NULL
,requirements VARCHAR2(500) NOT NULL
,date_in DATE NOT NULL
,emp_ID NUMBER(10) NOT NULL
CONSTRAINT fk_qual_emp
(FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE (emp_ID
;{
```

TABLE 5: LEAVE -----

```
) CREATE TABLE LEAVE
,leave_ID NUMBER(10) CONSTRAINT pk_leave PRIMARY KEY
,date_from DATE NOT NULL
,reason VARCHAR2(500) NOT NULL
,emp_ID NUMBER(10) NOT NULL
CONSTRAINT fk_leave_emp
(FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE (emp_ID
;{
```

TABLE 6: PAYROLL -----

```
) CREATE TABLE PAYROLL
,payroll_ID NUMBER(10) CONSTRAINT pk_payroll PRIMARY KEY
,date_paid DATE NOT NULL
,report VARCHAR2(500) NOT NULL
,total_amount NUMBER(12,2) NOT NULL
,emp_ID NUMBER(10) NOT NULL
,job_ID NUMBER(10) NOT NULL
,salary_ID NUMBER(10) NOT NULL
,(leave_ID NUMBER(10
CONSTRAINT fk_payroll_emp
,(FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE (emp_ID
CONSTRAINT fk_payroll_job
,(FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT (job_ID
CONSTRAINT fk_payroll_salary
,(FOREIGN KEY (salary_ID) REFERENCES SALARY_BONUS (salary_ID
CONSTRAINT fk_payroll_leave
```

```
(FOREIGN KEY (leave_ID) REFERENCES LEAVE      (leave_ID  
;(  

```

```
;CREATE SEQUENCE job_seq START WITH 1 INCREMENT BY 1  
;CREATE SEQUENCE salary_seq START WITH 1 INCREMENT BY 1  
;CREATE SEQUENCE emp_seq START WITH 1 INCREMENT BY 1  
;CREATE SEQUENCE qual_seq START WITH 1 INCREMENT BY 1  
;CREATE SEQUENCE leave_seq START WITH 1 INCREMENT BY 1  
;CREATE SEQUENCE payroll_seq START WITH 1 INCREMENT BY 1
```

```
(INSERT INTO JOB_DEPARTMENT (job_ID, job_dept, name, description, salary_range  
VALUES (job_seq.NEXTVAL, 'HR', 'Human Resources', 'Handles recruitment and employee  
;('relations', '1200-1500
```

```
(INSERT INTO JOB_DEPARTMENT (job_ID, job_dept, name, description, salary_range  
;('VALUES (job_seq.NEXTVAL, 'FIN', 'Finance', 'Manages company finances', '1500-2000
```

```
(INSERT INTO JOB_DEPARTMENT (job_ID, job_dept, name, description, salary_range  
VALUES (job_seq.NEXTVAL, 'IT', 'Information Technology', 'Maintains systems and  
;('networks', '2000-2500
```

```
(INSERT INTO JOB_DEPARTMENT (job_ID, job_dept, name, description, salary_range  
VALUES (job_seq.NEXTVAL, 'MKT', 'Marketing', 'Promotes products and services',  
;('1400-1800
```

```
(INSERT INTO JOB_DEPARTMENT (job_ID, job_dept, name, description, salary_range  
VALUES (job_seq.NEXTVAL, 'OPS', 'Operations', 'Oversees daily business processes',  
;('1300-1600
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, annual, bonus, job_ID  
;VALUES (salary_seq.NEXTVAL, 1200, 14400, 500, 1
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, annual, bonus, job_ID  
;VALUES (salary_seq.NEXTVAL, 1500, 18000, 600, 2
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, annual, bonus, job_ID  
;VALUES (salary_seq.NEXTVAL, 2000, 24000, 800, 3
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, annual, bonus, job_ID  
;VALUES (salary_seq.NEXTVAL, 1400, 16800, 400, 4
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, annual, bonus, job_ID  
;VALUES (salary_seq.NEXTVAL, 1300, 15600, 300, 5
```

```
INSERT INTO EMPLOYEE (emp_ID, fname, lname, gender, age, contact_add, emp_email,  
(emp_pass, job_ID, salary_ID  
VALUES (emp_seq.NEXTVAL, 'Ali', 'Hassan', 'M', 30, 'Muscat', 'ali.hassan@company.com',  
;('pass123', 1, 1
```

```
INSERT INTO EMPLOYEE (emp_ID, fname, lname, gender, age, contact_add, emp_email,  
(emp_pass, job_ID, salary_ID  
VALUES (emp_seq.NEXTVAL, 'Sara', 'Noor', 'F', 28, 'Sohar', 'sara.noor@company.com',  
;('pass123', 2, 2
```

```
INSERT INTO EMPLOYEE (emp_ID, fname, lname, gender, age, contact_add, emp_email,  
(emp_pass, job_ID, salary_ID  
VALUES (emp_seq.NEXTVAL, 'Omar', 'Said', 'M', 35, 'Nizwa', 'omar.said@company.com',  
;('pass123', 3, 3
```

```
INSERT INTO EMPLOYEE (emp_ID, fname, lname, gender, age, contact_add, emp_email,  
(emp_pass, job_ID, salary_ID  
VALUES (emp_seq.NEXTVAL, 'Layla', 'Khalid', 'F', 26, 'Muscat',  
;('layla.khalid@company.com', 'pass123', 4, 4
```

```
INSERT INTO EMPLOYEE (emp_ID, fname, lname, gender, age, contact_add, emp_email,  
(emp_pass, job_ID, salary_ID  
VALUES (emp_seq.NEXTVAL, 'Ahmed', 'Salim', 'M', 40, 'Sur', 'ahmed.salim@company.com',  
;('pass123', 5, 5
```

```
(INSERT INTO QUALIFICATION (qual_ID, position, requirements, date_in, emp_ID  
;(VALUES (qual_seq.NEXTVAL, 'Analyst', 'Bachelor's Degree', SYSDATE, 1
```

```
(INSERT INTO QUALIFICATION (qual_ID, position, requirements, date_in, emp_ID  
;(VALUES (qual_seq.NEXTVAL, 'Accountant', 'CPA Certification', SYSDATE, 2
```

```
(INSERT INTO QUALIFICATION (qual_ID, position, requirements, date_in, emp_ID  
;(VALUES (qual_seq.NEXTVAL, 'Developer', 'Java Certification', SYSDATE, 3
```

```
(INSERT INTO QUALIFICATION (qual_ID, position, requirements, date_in, emp_ID  
;(VALUES (qual_seq.NEXTVAL, 'Marketer', 'Digital Marketing Diploma', SYSDATE, 4
```

```
(INSERT INTO QUALIFICATION (qual_ID, position, requirements, date_in, emp_ID  
;(VALUES (qual_seq.NEXTVAL, 'Manager', 'MBA Degree', SYSDATE, 5
```

```
(INSERT INTO LEAVE (leave_ID, date_from, reason, emp_ID  
;(VALUES (leave_seq.NEXTVAL, TO_DATE('2026-06-01','YYYY-MM-DD'), 'Medical', 1
```

```
(INSERT INTO LEAVE (leave_ID, date_from, reason, emp_ID
```

```

;(VALUES (leave_seq.NEXTVAL, TO_DATE('2026-06-05','YYYY-MM-DD'), 'Vacation', 2

(INSERT INTO LEAVE (leave_ID, date_from, reason, emp_ID
VALUES (leave_seq.NEXTVAL, TO_DATE('2026-06-10','YYYY-MM-DD'), 'Family
;(Emergency', 3

(INSERT INTO LEAVE (leave_ID, date_from, reason, emp_ID
;(VALUES (leave_seq.NEXTVAL, TO_DATE('2026-06-12','YYYY-MM-DD'), 'Training', 4

(INSERT INTO LEAVE (leave_ID, date_from, reason, emp_ID
;(VALUES (leave_seq.NEXTVAL, TO_DATE('2026-06-15','YYYY-MM-DD'), 'Conference', 5

INSERT INTO PAYROLL (payroll_ID, date_paid, report, total_amount, emp_ID, job_ID,
(salary_ID, leave_ID
;(VALUES (payroll_seq.NEXTVAL, SYSDATE, 'Monthly Payroll', 12500, 1, 1, 1, NULL

INSERT INTO PAYROLL (payroll_ID, date_paid, report, total_amount, emp_ID, job_ID,
(salary_ID, leave_ID
;(VALUES (payroll_seq.NEXTVAL, SYSDATE, 'Monthly Payroll', 18600, 2, 2, 2, 2

INSERT INTO PAYROLL (payroll_ID, date_paid, report, total_amount, emp_ID, job_ID,
(salary_ID, leave_ID
;(VALUES (payroll_seq.NEXTVAL, SYSDATE, 'Monthly Payroll', 24800, 3, 3, 3, NULL

INSERT INTO PAYROLL (payroll_ID, date_paid, report, total_amount, emp_ID, job_ID,
(salary_ID, leave_ID
;(VALUES (payroll_seq.NEXTVAL, SYSDATE, 'Monthly Payroll', 17200, 4, 4, 4, 4

INSERT INTO PAYROLL (payroll_ID, date_paid, report, total_amount, emp_ID, job_ID,
(salary_ID, leave_ID
;(VALUES (payroll_seq.NEXTVAL, SYSDATE, 'Monthly Payroll', 15900, 5, 5, 5, 5

```

Task 2: Conditional SELECT Queries

:Write SELECT queries for each of the following requirements

- List all employees whose age is between 25 and 40, ordered by last name ascending)
 - Retrieve all payroll records where total_amount exceeds 5000, showing employee name) and department
 - Find all employees who have taken leave with reason containing the word 'sick')
- ((case-insensitive

(.d) List all departments that have no employees assigned. (Use outer join or NOT EXISTS)

.A--

```
SELECT emp_ID, fname, lname, age
FROM EMPLOYEE
WHERE age BETWEEN 25 AND 40
;ORDER BY lname ASC
```

.B--

```
,SELECT p.payroll_ID
,e.fname || ' ' || e.lname AS employee_name
,d.job_dept AS department
p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT d ON p.job_ID = d.job_ID
;WHERE p.total_amount > 5000
```

.C--

```
SELECT DISTINCT e.emp_ID, e.fname, e.lname, l.reason
FROM EMPLOYEE e
JOIN LEAVE l ON e.emp_ID = l.emp_ID
;WHERE LOWER(l.reason) LIKE '%sick
```

.D--

```
SELECT d.job_ID, d.job_dept, d.name
FROM JOB_DEPARTMENT d
LEFT JOIN EMPLOYEE e ON d.job_ID = e.job_ID
;WHERE e.emp_ID IS NULL
```

Aggregation Functions 04

Task 1: Basic Aggregation

:Write queries using standard aggregate functions

.a) Total number of employees in each department)


```
,SELECT d.job_ID
,d.job_dept
COUNT(e.emp_ID) AS total_employees
FROM JOB_DEPARTMENT d
LEFT JOIN EMPLOYEE e ON d.job_ID = e.job_ID
GROUP BY d.job_ID, d.job_dept
;ORDER BY d.job_dept
```

.b) Minimum, maximum, and average salary (amount) across all salary records)

```
,SELECT MIN(amount) AS min_salary
,MAX(amount) AS max_salary
AVG(amount) AS avg_salary
;FROM SALARY_BONUS
```

.c) Total bonus paid out across the entire company)

```
SELECT SUM(bonus) AS total_bonus
;FROM SALARY_BONUS
```

Task 2: GROUP BY with HAVING

:Write GROUP BY queries with HAVING filters

.a) List departments where the average employee age exceeds 30)

```
,SELECT d.job_ID
,d.job_dept
AVG(e.age) AS avg_age
FROM JOB_DEPARTMENT d
JOIN EMPLOYEE e ON d.job_ID = e.job_ID
GROUP BY d.job_ID, d.job_dept
;HAVING AVG(e.age) > 30
```

.b) Show all job titles where more than 2 employees share that qualification position)

```
,SELECT q.position
COUNT(e.emp_ID) AS num_employees
```

```

FROM QUALIFICATION q
JOIN EMPLOYEE e ON q.emp_ID = e.emp_ID
GROUP BY q.position
;HAVING COUNT(e.emp_ID) >2

```

c) Find months (from PAYROLL.date) where the total payroll amount exceeds 20,000)

```

,SELECT TO_CHAR(p.date_paid, 'YYYY-MM') AS payroll_month
SUM(p.total_amount) AS monthy_totla
FROM PAYROLL p
('GROUP BY TO_CHAR(p.date_paid, 'YYYY-MM
HAVING SUM(p.total_amount) > 20000
;ORDER BY payroll_month

```

Joins 05

Task 1: INNER JOIN — Employee Full Profile

.Write a query using INNER JOINS to retrieve a complete employee profile

Columns: emp_ID, full name (fname || ' ' || lname), department name, job title (from QUALIFICATION), salary

.amount, latest leave date

.Filter: only employees who have both a payroll record AND a qualification record

```

,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
,d.name AS department_name
,q.position AS job_title
,s.amount AS salary_amount
MAX(l.date_from) AS latest_leave_date
FROM EMPLOYEE e
INNER JOIN JOB_DEPARTMENT d ON e.job_id = d.job_ID
INNER JOIN QUALIFICATION q ON e.emp_ID = q.emp_ID
INNER JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
INNER JOIN PAYROLL p ON e.emp_ID = p.emp_ID
LEFT JOIN LEAVE l ON e.emp_id = l.emp_ID
;GROUP BY e.emp_ID, e.fname, e.lname , d.name, q.position , s.amount

```

Task 2: LEFT OUTER JOIN — Missing Records

:Use LEFT OUTER JOINS to find gaps in data

.(a) List all employees who have never taken any leave (no matching LEAVE record)

```
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
e.job_ID
FROM EMPLOYEE e
LEFT OUTER JOIN LEAVE l ON e.emp_ID = l.emp_ID
;WHERE l.leave_ID IS NULL
```

.b) List all departments that have no salary/bonus records associated with them)

```
,SELECT d.job_ID
,d.job_dept
d.name
FROM JOB_DEPARTMENT d
LEFT OUTER JOIN SALARY_BONUS s ON d.job_ID = s.job_ID
;WHERE s.salary_ID IS NULL
```

Subqueries 06

Task 1: Single-Row Subquery

:Write queries using single-row subqueries

a) Find all employees whose salary is greater than the average salary of the entire company

```
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
s.amount AS salary_amount
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
) < WHERE s.amount
(SELECT AVG(amount
FROM SALARY_BONUS
(
;ORDER BY s.amount DESC
```

b) Retrieve the department with the highest total payroll amount)

```
,SELECT d.job_ID
,d.job_dept
,d.name
(SELECT SUM(p.total_amount)
FROM PAYROLL p
WHERE p.job_ID = d.job_ID) AS dept_totl
FROM JOB_DEPARTMENT d
) = WHERE d.job_ID
SELECT job_ID
FROM PAYROLL
GROUP BY job_ID
ORDER BY SUM(total_amount) DESC
FETCH FIRST 1 ROW ONLY

;{
```

Task 2: Multi-Row Subquery with IN / ANY / ALL

**a) List all employees who work in departments that have at least one salary record)
 .with a bonus greater than 500
 .Use IN**

```
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
,e.job_ID
FROM EMPLOYEE e
) WHERE e.job_ID IN
SELECT DISTINCT job_ID
FROM SALARY_BONUS
WHERE bonus > 500

;{
```

**b) Find employees whose salary is greater than ALL salaries in the 'Maintenance')
 .department. Use ALL**

```
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
```

```

s.amount AS salary_amount
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
) WHERE s.amount > ALL
SELECT s2.amount
FROM EMPLOYEE e2
JOIN JOB_DEPARTMENT d ON e2.job_ID = d.job_ID
JOIN SALARY_BONUS s2 ON e2.salary_ID = s2.salary_ID
'WHERE d.job_dept = 'Maintenance

;(

```

**c) Find employees whose salary is greater than ANY salary in the 'HR' department.)
 .Use ANY**

```

,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
s.amount AS salary_amount
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
) WHERE s .amount > ANY
SELECT s2.amount
FROM EMPLOYEE e2
JOIN JOB_DEPARTMENT d ON e2.job_ID = D.JOB_ID
JOIN SALARY_BONUS s2 ON e2.salary_ID = s2.salary_ID
'WHERE d.job_dept = 'HR

;(

```

Views 07

Task 1: Simple Read-Only View

:Create a view named VW_EMPLOYEE_SUMMARY that returns

emp_ID, full name, gender, age, department name, and job title (from

.(QUALIFICATION

:After creating it

a) Query the view to list all female employees over 30)

b) Try to INSERT a row through this view and document the Oracle error message you receive

```
CREATE OR REPLACE VIEW VW_EMPLOYEE_SUMMARY AS
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
,e.gender
,e.age
,d.name AS department_name
,q.position AS job_title
FROM EMPLOYEE e
JOIN JOB_DEPARTMENT d ON e.job_ID = d.job_ID
;JOIN QUALIFICATION q ON e.emp_ID = q.emp_ID
```

--

```
SELECT emp_ID, full_name , department_name , job_title
FROM VW_EMPLOYEE_SUMMARY
'WHERE gender = 'F
;AND age > 30
```

.2--

```
INSERT INTO VW_EMPLOYEE_SUMMARY (emp_ID , full_name , gender , age ,
(department_name , job_title
(' VALUES(979,'Maha' , 'M' ,24 , 'Finance' , 'Analyst
```

: More Details

[/https://docs.oracle.com/error-help/db/ora-00942](https://docs.oracle.com/error-help/db/ora-00942)

- Error starting at line : 571 in command

```
INSERT INTO VW_EMPLOYEE_SUMMARY (emp_ID , full_name , gender , age ,
(department_name , job_title
(' VALUES(979,'Maha' , 'F' ,24 , 'Finance' , 'Analyst
Error at Command Line : 571 Column : 34
```

- Error report

SQL Error: ORA-01779: cannot modify a column which maps to a non key-preserved table

<https://docs.oracle.com/error-help/db/ora-01779/01779.00000> - "cannot modify a column
"which maps to a non key-preserved table

Cause: An attempt was made to insert or update columns of a join view which*
map to a non-key-preserved table

Action: Modify the underlying base tables directly*

: More Details

[/https://docs.oracle.com/error-help/db/ora-01779](https://docs.oracle.com/error-help/db/ora-01779)

Task 2: Payroll Dashboard View

Create a view named VW_PAYROLL_DASHBOARD that returns for each payroll record payroll_ID, employee full name, department, salary amount, bonus, leave reason, payroll date, and total_amount
Query the view to find the top 5 payroll records by total_amount

```
CREATE OR REPLACE VIEW VW_PAYROLL_DASHBOARD AS
,SELECT p.payroll_ID
,e.fname || ' ' || e.lname AS employee_name
,d.name AS department_name
,s.amount AS salary_amount
,s.bonus
,l.reason AS leave_reason
,p.date_paid AS payroll_date
p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT d ON p.job_ID = d.job_ID
JOIN SALARY_BONUS s ON p.salary_ID = s.salary_ID
;LEFT JOIN LEAVE l ON p.leave_ID = l.leave_ID
```

```
,SELECT payroll_ID
,employee_name
,department_name
,salary_amount
,bonus
,leave_reason
,payroll_date
total_amount
FROM VW_PAYROLL_DASHBOARD
ORDER BY total_amount DESC
;FETCH FIRST 5 ROWS ONLY
```

Stored Procedures & Functions 08

Task 1: Procedure — Add New Employee

Create a stored procedure named SP_ADD_EMPLOYEE with IN parameters for all **EMPLOYEE** columns (except

.(emp_ID which is auto-generated from a sequence

:The procedure must

.(Validate that emp_email is not already in use (raise an exception if duplicate •

.Insert the new employee record •

.Print a confirmation message using DBMS_OUTPUT.PUT_LINE •

Test by calling the procedure twice with the same email to confirm the exception fires

```
) CREATE TABLE SALARY_BONUS
,salary_ID  NUMBER PRIMARY KEY
,amount    NUMBER
,bonus     NUMBER
,annual    NUMBER NOT NULL
;{(
```

```
(INSERT INTO SALARY_BONUS (salary_ID, amount, bonus, annual
;(VALUES (222, 1500, 200, 1700
```

```
;SELECT job_ID, name FROM JOB_DEPARTMENT
```

Task 1: Procedure--

```
) CREATE OR REPLACE PROCEDURE SP_ADD_EMPLOYEE
,p_job_ID      IN EMPLOYEE.job_ID%TYPE
,p_salary_ID   IN EMPLOYEE.salary_ID%TYPE
,p_fname      IN EMPLOYEE.fname%TYPE
,p_lname      IN EMPLOYEE.lname%TYPE
,p_gender      IN EMPLOYEE.gender%TYPE
,p_age        IN EMPLOYEE.age%TYPE
,p_contact_add IN EMPLOYEE.contact_add%TYPE
,p_emp_email   IN EMPLOYEE.emp_email%TYPE
p_emp_pass    IN EMPLOYEE.emp_pass%TYPE
```

```
AS (
;v_count NUMBER
```

```
BEGIN
```

```
(* )SELECT COUNT
```



```
INTO v_count
FROM EMPLOYEE
;WHERE emp_email = p_emp_email
```

```
IF v_count > 0 THEN
;(RAISE_APPLICATION_ERROR(-20001, 'Email already exists: ' || p_emp_email
;END IF
```

```
) INSERT INTO EMPLOYEE
emp_ID , job_ID , salary_ID, fname , lname , gender , age,contact_add , emp_email ,
emp_pass
(
) VALUES
emp_seq.NEXTVAL , p_job_ID , p_salary_ID , p_fname , p_lname , p_gender , p_age ,
p_contact_add ,p_emp_email , p_emp_pass
;{
```

```
DBMS_OUTPUT.PUT_LINE('Employee ' || p_fname || ' ' || p_lname || ' added
;('successfully
;END
/
```

```
BEGIN
)SP_ADD_EMPLOYEE
,p_job_ID      => 1
,p_salary_ID   => 222
,p_fname       => 'Maha
,p_lname       => 'Ali
,p_gender      => 'F
,p_age         => 28
,p_contact_add => 'Muscat ,Oman
,p_emp_email   => 'Maha.ali@gmail.com
,p_emp_pass    => 'securepass

;{
;END
/
```

```
-----
BEGIN
)SP_ADD_EMPLOYEE
,p_job_ID      => 1
,p_salary_ID   => 222
,p_fname       => 'Maha
,p_lname       => 'Ali
```

```
,p_gender      => 'F
,p_age         => 29
,p_contact_add => 'Muscat ,Oman
,p_emp_email   => 'Maha.ali@gmail.com
,p_emp_pass    => 'anotherpass

;(
;END
/
```

OUTPUT: ORA-20001: Email already exists: Maha.ali@gmail.com

Task 2: Function — Calculate Net Salary

Create a function named FN_NET_SALARY that accepts emp_ID as input and returns the employee's net salary

.amount + bonus) as a NUMBER)

Use it in a SELECT query to display all employees with their calculated net salary, ordered highest to lowest

```
)CREATE OR REPLACE FUNCTION FN_NET_SALARY
p_emp_ID IN EMPLOYEE.emp_ID%TYPE
RETURN NUMBER (
IS
;v_net_salary NUMBER

BEGIN
SELECT s.amount + s.bonus
INTO v_net_salary
FROM SALARY_BONUS s
JOIN EMPLOYEE e ON e.salary_ID = s.salary_ID
; WHERE e.emp_ID = p_emp_ID
;RETURN v_net_salary
EXCEPTION
WHEN NO_DATA_FOUND THEN
;RETURN 0
;END
/
```

```
,SELECT e.emp_ID
,e.fname || ' ' || e.lname AS full_name
FN_NET_SALARY(e.emp_ID) AS net_salary
FROM EMPLOYEE e
;ORDER BY net_salary DESC
```

Triggers 09

Task 1: BEFORE INSERT — Auto-Assign emp_ID

.Create a BEFORE INSERT trigger named TRG_EMP_ID on the EMPLOYEE table

If the inserted emp_ID is NULL, the trigger should populate it from a sequence

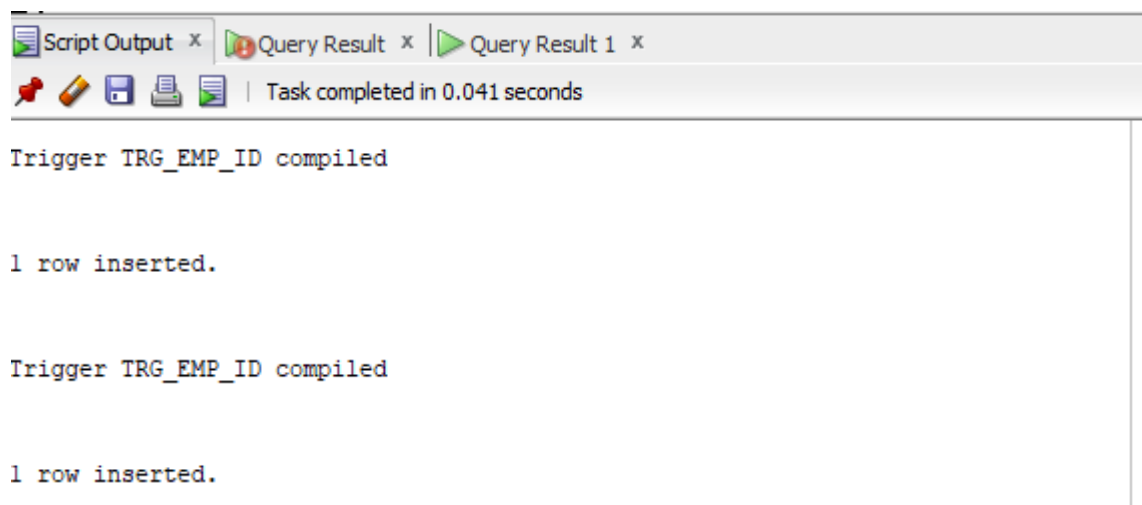
.(EMP_SEQ.NEXTVAL

Test: INSERT an employee without specifying emp_ID and verify it was auto-assigned

TASK 1 --

```
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
IF :NEW.emp_ID IS NULL THEN
SELECT emp_seq.NEXTVAL
INTO :NEW.emp_ID
;FROM dual
;END IF
;END
/
```

```
,INSERT INTO EMPLOYEE( job_ID, salary_ID, fname, lname, gender, age
contact_add, emp_email, emp_pass
(
VALUES
'Maha', 'Ali' , 'F' , 28,'Muscat ,Oman','Maha.ali@gmail.com','securepass' , 222 , 1)
;{
```



```
SELECT emp_ID , fname , lname, emp_email
FROM EMPLOYEE
;WHERE emp_email = 'maha.auto2@gmail.com
```

EMP_ID	FNAME	LNAME	EMP_EMAIL
1	30 Maha	Ali	maha.auto@gmail.com

Task 2: AFTER INSERT — Welcome Log

.(Create a table EMPLOYEE_LOG(log_id, emp_id, action, log_timestamp)
 Create an AFTER INSERT trigger named TRG_EMP_WELCOME_LOG on EMPLOYEE that
 inserts a log row with
 .action = 'NEW HIRE' whenever a new employee is added
 Test by inserting 2 employees and querying EMPLOYEE_LOG

```
) CREATE TABLE EMPLOYEE_LOG
,log_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
,emp_id      NUMBER
,(action     VARCHAR2(50
log_timestamp DATE
;
(
```

```
Table EMPLOYEE_LOG created.
```

```
CREATE OR REPLACE TRIGGER TRG_EMP_WELCOME_LOG
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
(INSERT INTO EMPLOYEE_LOG (emp_id , action , log_timestamp
;VALUES(:new.emp_ID, 'NEW HIRE' , SYSDATE
;END
/
```

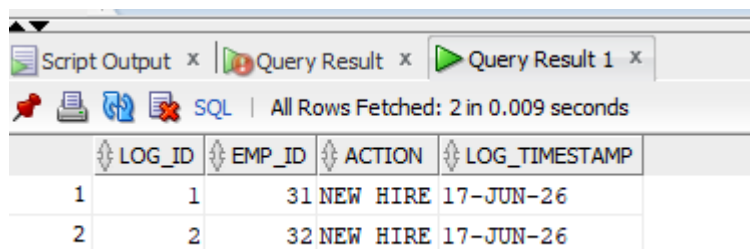
```
Trigger TRG_EMP_WELCOME_LOG compiled
```

```
INSERT INTO EMPLOYEE ( job_ID, salary_ID, fname, lname, gender, age, contact_add,
emp_email, emp_pass
VALUES (1, 222, 'Sara', 'Khamees', 'F', 30, 'Muscat, Oman', 'sara.kha@gmail.com', (
;('pass123
```

```
INSERT INTO EMPLOYEE ( job_ID, salary_ID, fname, lname, gender, age, contact_add,
emp_email, emp_pass
VALUES (2, 333, 'Hameed', 'Salim', 'M', 35, 'Muscat, Oman', 'Hammed.Sal@gmail.com', (
;('pass456
```

```
;COMMIT
```

```
SELECT log_id , emp_id , action, log_timestamp
FROM EMPLOYEE_LOG
;ORDER BY log_id
```



The screenshot shows a database query result window with the following data:

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	1	31 NEW HIRE	17-JUN-26
2	2	32 NEW HIRE	17-JUN-26

Oracle Scheduler Jobs 10

Task 1: Simple One-Time Scheduler Job

Create a one-time DBMS_SCHEDULER job named JOB_GREET_EMPLOYEES that runs 2 minutes from now and executes an anonymous PL/SQL block that prints 'Payroll System Initialized' to DBMS_OUTPUT and inserts a record into EMPLOYEE_LOG

```
BEGIN
)DBMS_SCHEDULER.CREATE_JOB
,'job_name => 'JOB_GREET_EMPLOYEES
,'job_type => 'PLSQL_BLOCK
,'job_action => 'BEGIN ... END
,'start_date => SYSTIMESTAMP + INTERVAL "2" MINUTE
```

```
,(enabled => TRUE
```

```
;(
```

```
;END
```

.After the job runs, query USER_SCHEDULER_JOB_LOG to confirm execution status

```
BEGIN
```

```
) DBMS_SCHEDULER.CREATE_JOB
```

```
, 'job_name' => 'JOB_GREET_EMPLOYEES
```

```
, 'job_type' => 'PLSQL_BLOCK
```

```
, 'job_action'
```

```
BEGIN
```

```
Print to DBMS_OUTPUT --
```

```
;"DBMS_OUTPUT.PUT_LINE("Payroll System Initialized
```

```
Insert into EMPLOYEE_LOG --
```

```
(INSERT INTO EMPLOYEE_LOG (emp_id, action, log_timestamp
```

```
;VALUES (NULL, "SYSTEM INIT", SYSDATE
```

```
);END
```

```
, 'start_date' => SYSTIMESTAMP + INTERVAL '2' MINUTE
```

```
, 'enabled' => TRUE
```

```
;(
```

```
;END
```

```
/
```

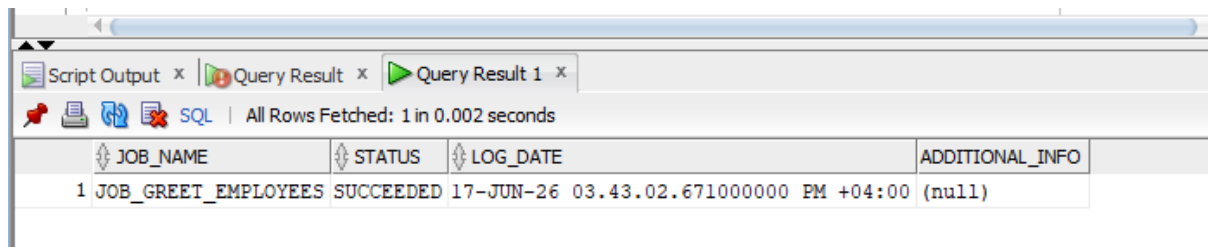
```
PL/SQL procedure successfully completed.
```

```
SELECT job_name, status, log_date, additional_info
```

```
FROM USER_SCHEDULER_JOB_LOG
```

```
WHERE job_name = 'JOB_GREET_EMPLOYEES
```

```
ORDER BY log_date DESC
```



The screenshot shows the SQL Developer interface with a query result window. The window title is "Query Result 1". It displays a table with the following data:

	JOB_NAME	STATUS	LOG_DATE	ADDITIONAL_INFO
1	JOB_GREET_EMPLOYEES	SUCCEEDED	17-JUN-26 03.43.02.671000000 PM +04:00	(null)

Task 2: Recurring Job — Daily Leave Report

Create a recurring DBMS_SCHEDULER job named JOB_DAILY_LEAVE_REPORT that:

- Runs every day at 07:00 AM.
- Calls the procedure SP_PROCESS_PAYROLL or simply counts leave records taken that day and inserts a summary into EMPLOYEE_LOG.

Use a repeat interval:

FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0. Show the job definition using:

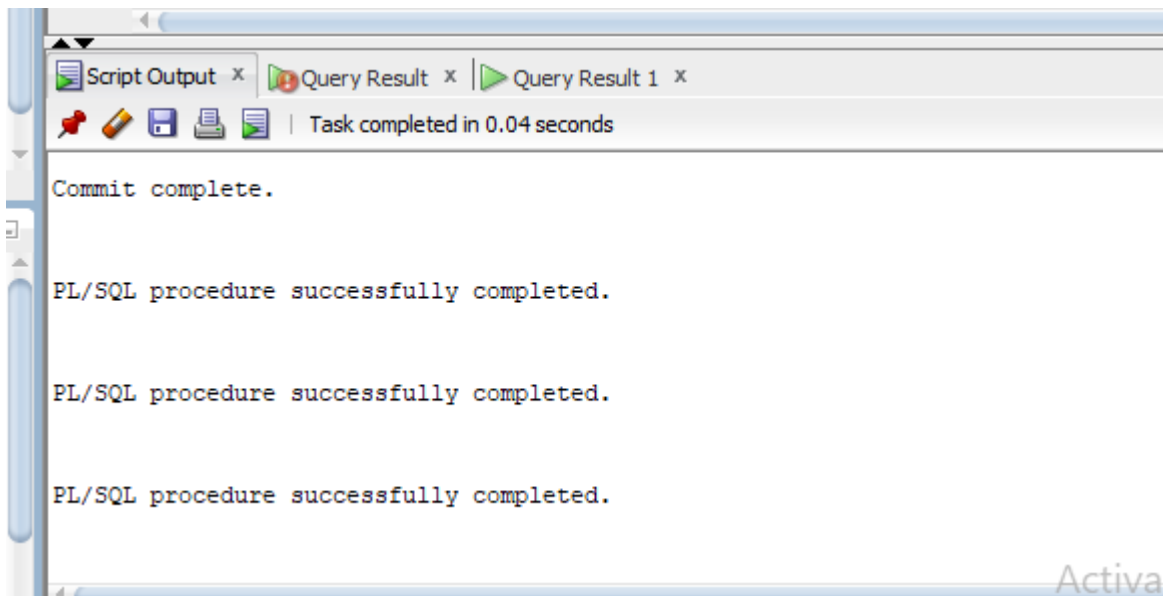
SELECT * FROM USER_SCHEDULER_JOBS WHERE JOB_NAME = 'JOB_DAILY_LEAVE_REPORT';

BEGIN

```
DBMS_SCHEDULER.CREATE_JOB (
  job_name      => 'JOB_DAILY_LEAVE_REPORT',
  job_type      => 'PLSQL_BLOCK',
  job_action    => '
    BEGIN
      -- Option 1: Call your payroll procedure
      -- SP_PROCESS_PAYROLL;

      -- Option 2: Count today"s leave records and log
      DECLARE
        v_leave_count NUMBER;
      BEGIN
        SELECT COUNT(*)
        INTO v_leave_count
        FROM LEAVE
        WHERE TRUNC(leave_date) = TRUNC(SYSDATE);

        INSERT INTO EMPLOYEE_LOG (emp_id, action, log_timestamp)
        VALUES (NULL, "LEAVE SUMMARY: " || v_leave_count, SYSDATE);
      END;
    END;';
  start_date    => SYSTIMESTAMP,
  repeat_interval => 'FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0',
  enabled       => TRUE
);
END;
/
```



```
SELECT job_name, enabled, repeat_interval, job_type, job_action
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_DAILY_LEAVE_REPORT';
```

Script Output x Query Result x Query Result 1 x

All Rows Fetched: 1 in 0.13 seconds

JOB_NAME	ENABLED	REPEAT_INTERVAL	JOB_TYPE	JOB_ACTION
1 JOB_DAILY_LEAVE_REPORT	TRUE	FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0	PLSQL_BLOCK	BEGIN -- Option 1: Call